Assignment 2 Report

Author: Luka Karanovic

Date: October 13th, 2026

Overview

This report outlines my process of making a decision tree predictive model for a modified version of the German Creditor Dataset. The German Creditor dataset contains a set of bank customers described by a set of attributes, and is generally used to classify these customers as either good or bad credit risks. The data set is downloaded from the OpenML website, and in the original format provided by Prof. Hofmann. The modified version of the data set removed most descriptive attributes and only has the following: "purpose", "employment", "housing", "job", "foreign_worker". The decision tree uses these five descriptive attributes to build a decision tree to predict the ‘Credit_Risk’ target attribute.

The report goes over the data exploration and preprocessing of these five attributes, the decision tree training process, an analysis of the constructed decision tree, and the testing of the decision tree, along with possible limitations of the model.

Data Exploration

This section outlines a summary of the data for each of the selected attributes we are using to build our decision tree model. The information from this section is based on the data collected from assignment 1.

Recategorizing the ‘purpose’ attribute

Note that all of the attributes are categorical so they are suitable for a decision tree classifier, however, the attribute purpose has a lot of different categories so I decided to group some of the values together based on the type of reason they are applying for credit for.

- Vehicular purchase: {‘used car’, ‘new car’, ‘repairs’}
- Home appliances: {'furniture/equipment', 'radio/tv', 'domestic appliance'}
- Education/business: {'education', 'retraining', 'business'}
- Other: {‘other’}

The groupings are based on the type of purchase it is. This is because the type of purchase is generally within the same money range and also has a similar demographic of people looking to buy it.

purpose

Semantic Meaning: The applicant's purpose for taking out credit from the bank.

Data Type: Nominal

Data Item Count: 1000

of Missing Values: 0

% of Missing Values: 0%

Cardinality of attribute (# of unique values): 10

BEFORE GROUPING: List of all possible values and the frequency of each value:

Value	**Frequency**
radio/tv	280
'new car'	234
furniture/equipment	181
'used car'	103
business	97
education	50
repairs	22
'domestic appliance'	12
other	12
retraining	9

AFTER GROUPING: List of all possible values and the frequency of each value:

- *Note: Used both training and testing dataset for this analysis*

Cardinality of attribute (# of unique values): 4

Value	**Frequency**
home_appliances	473
vehicle	359
education_business	156
other	12

---

employment

Semantic Meaning: The number of years the applicant has been employed at their current job.

Data Type: Ordinal

Data Item Count: 1000

of Missing Values: 0

% of Missing Values: 0%

Cardinality of attribute (# of unique values): 5

List of all possible values and the frequency of each value:

Value	Frequency
'1<=X<4'	339
'>=7'	253
'4<=X<7'	174
'<1'	172
unemployed	62

---

housing

Semantic Meaning: The applicant's current home-owning status. It indicates whether the applicant owns, rents, or lives in a home 'for free'.

Data Type: Ordinal

Data Item Count: 1000

of Missing Values: 0

% of Missing Values: 0%

Cardinality of attribute (# of unique values): 3

List of all possible values and the frequency of each value:

Value	Frequency
own	713
rent	179
'for free'	108

---

job

Semantic Meaning: Classifies applicants based on economic level rather than specific occupations. This column ranges from 'unskilled resident' (low economic tier) to professional tiers like 'high qualif./mgmt'.

Data Type: Ordinal (can be debated on the order)

Data Item Count: 1000

of Missing Values: 0

% of Missing Values: 0%

Cardinality of attribute (# of unique values): 4

List of all possible values and the frequency of each value:

Value	Frequency
skilled	630
'unskilled resident'	200
'high qualif/self emp/mgmt'	148
'unemp/unskilled non res'	22

---

foreign_worker

Semantic Meaning: Whether or not the applicant is a foreign worker living in Germany.

Data Type: Nominal

Data Item Count: 1000

of Missing Values: 0

% of Missing Values: 0%

Cardinality of attribute (# of unique values): 2

List of all possible values and the frequency of each value:

Value	Frequency
yes	963
no	37

One-Hot Encoding

A big issue I ran in to is that Sci-kit Learn's `DecisionTreeClassifier` class requires numerical value inputs. All of the input data we have is in the form of a 'string' as the attributes are all categorical.

I used **One-Hot Encoding**, which is a data preprocessing technique used in machine learning to convert categorical data (text labels, names, or discrete groups) into a numerical binary format that algorithms like `DecisionTreeClassifier` can understand.

- For example, `purpose`, which had 4 distinct values, was turned into 4 boolean columns:
  - `purpose_vehicle`, `purpose_home_appliances`, `purpose_education_business`, and `purpose_other`
  - The values in each of these columns is either 0 or 1:
    - 0 if the object's `purpose` value was previously not equal to that value
    - 1 if the object's `purpose` value was previously equal to that value
- This process was done for each column before being put into scikit-learn's `DecisionTreeClassifier` model.

Decision Tree Training Process

The first thing I did was extract the data from the Excel spreadsheet and recategorize the `purpose` attribute using `pandas` and `numpy`. I made sure to make a separate `pandas DataFrame` for the training data and the testing data.

One thing I had to do was combine the attributes of the training and testing datasets before One-Hot Encoding as there was a chance that some attribute values would be in one training set instead of the other, leading to the encoding of the training and testing datasets being inconsistent.

- Example: Consider the `housing` attribute: if the training data had all 3 values (own, rent, 'for free') the encoding would make 3 columns for this attribute and build the tree using them.
- What if the testing data set only had values own and rent for `housing`? Then the encoding would not make a `housing_'for free'` column and if a testing object encountered a node that used that column, it would cause an error (to my knowledge)
    - ***This means query objects would have to be One-Hot Encoded by being combined with the training dataset's columns prior to navigating the tree.***

Additionally, after One-Hot Encoding, it made the tree pretty difficult to read. Since it turned each value into a feature and used 0.5 as a threshold (so we had nodes named `employment_'>=7' <= 0.5` and `employment_unemployed <= 0.5`, etc.),

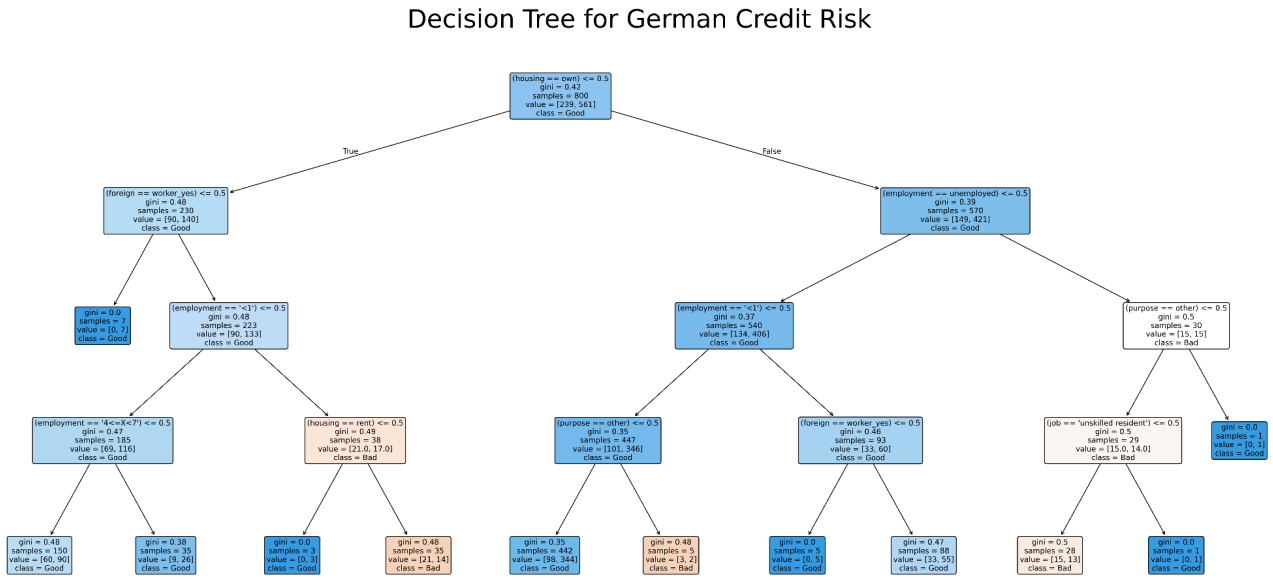
- While I could not strip out the '`<= 05`' on each node, I made a loop that converted the column names to be more readable:
    - An example conversion: `purpose_vehicle` to `(purpose == vehicle)`
    - Each tree node reads like `(feature == value) <= 0.5` where the left branch means that object had a value of 0 (`feature != value`, or false), and the right branch means that object had a value of 1 (`feature == value`, or true)

After that, I ended up with my first decision tree:

Decision Tree Result

Visual Diagram (generated by matplotlib)

- Ignore the 'True' and 'False' labels `matplotlib` for some reason put on the root node branches, they are confusing (`<= 0.5` being true means that the equality in the node is false)



Decision Tree Rules (in text):

```
|--- (housing == own) is false
|   |--- (foreign == worker_yes) is false
|   |   |--- class: good
|   |--- (foreign == worker_yes) is true
|   |   |--- (employment == '<1') is false
|   |   |   |--- (employment == '4<=X<7') is false
|   |   |   |   |--- class: good
|   |   |   |--- (employment == '4<=X<7') is true
|   |   |   |   |--- class: good
|   |   |--- (employment == '<1') is true
|   |   |   |--- (housing == rent) is false
|   |   |   |   |--- class: good
|   |   |   |--- (housing == rent) is true
|   |   |   |   |--- class: bad
|--- (housing == own) is true
|   |--- (employment == unemployed) is false
|   |   |--- (employment == '<1') is false
|   |   |   |--- (purpose == other) is false
|   |   |   |   |--- class: good
|   |   |   |--- (purpose == other) is true
|   |   |   |   |--- class: bad
|   |   |--- (employment == '<1') is true
|   |   |   |--- (foreign == worker_yes) is false
|   |   |   |   |--- class: good
|   |   |   |--- (foreign == worker_yes) is true
|   |   |   |   |--- class: good
|   |--- (employment == unemployed) is true
|   |   |--- (purpose == other) is false
|   |   |   |--- (job == 'unskilled resident') is false
```

```
|   |   |   |   |--- class: bad
|   |   |   |--- (job == 'unskilled resident') is true
|   |   |   |   |--- class: good
|   |   |--- (purpose == other) is true
|   |   |   |--- class: good
```

Decision Tree Analysis

Algorithm Used: CART

Here is a summary of how Scikit-learn's `DecisionTreeClassifier` algorithm works, it uses the CART algorithm and can **only build a binary tree**:

Impurity measurement method: using Gini Index (could use Entropy, but I went with Gini)
Stopping Criteria: Stop splitting when a node:

- reaches a pure state (all data objects have same class)
- hits a maximum depth limit (I specified my tree to have a maximum depth of 4)
- set of features left to test is empty: return a leaf node tree with the majority class of the dataset as its classification.
- dataset is empty: return a leaf node tree with the majority class of the dataset at the parent node that made the recursive call.

Since `DecisionTreeClassifier` can only produce a binary tree, it is restricted to binary decisions only, which I will talk about how that may be limiting later.

1. For each feature, test every possible binary split point (yes/no question or threshold value).
2. Calculate the weighted Gini impurity of the left and right child nodes resulting from each split.
    - This is the same as the 'information gain using Gini index' in our lecture slides with k=2 for the sum.
3. Choose the feature and split point that yields the lowest impurity (or highest information gain).
4. Partition the parent node's data into two child nodes (left and right) based on the chosen rule.
5. Apply steps 2 through 4 recursively to each new child node.
6. Assign a prediction to each leaf node (majority class)

Node Selection Explanation

`housing` was used as the first descriptive attribute placed in the root node because the Gini gain for whether or not the value of `housing` was equal to `own` or not was the highest of all candidate decisions.

The leaf node values were displayed above in the graph and text tree, the value (’Good’ or ‘Bad’) was determined by a majority vote:

- If a leaf node has a label ‘Bad’, it was because more data objects in that leaf node had a value of ‘Bad’ than ‘Good’
- If a leaf node has a label ‘Good’, it was because more data objects in that leaf node had a value of ‘Good’ than ‘Bad’

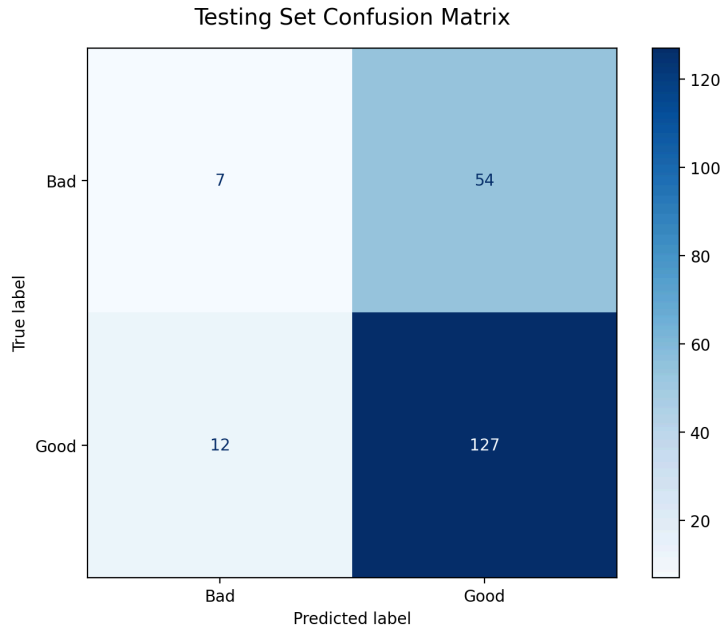
Testing

As said above, each of the testing data objects was one-hot encoded with matching columns from the training dataset, ensuring it had all of the same columns.

- From above: *“This means query objects would have to be One-Hot Encoded by being combined with the training dataset’s columns prior to navigating the tree.”*

Then, each testing object was put at the root node, and then navigated through the binary decision tree until it hit a leaf node. After it hit a leaf node, it was given a predicted value based on the label at the leaf node (‘Good’ or ‘Bad’).

Comparison is simple, I built a confusion matrix with the columns (x-axis) being the label my model predicts and the rows (y-axis) being the label that the object actually had.



The squares going down diagonally from the top-left are accurate predictions and the other squares are inaccurate predictions.

- As you can see, the model predicted correctly for (127+7)/200 (67%) of objects, and incorrectly for (12+54)/200 (33%) of objects.
- The model was quite bad at predicting when the credit risk was actually ‘Bad’, incorrectly classifying it as ‘Good’ 54 times.

The cost matrix presented in the Excel spreadsheet's 'Metadata' sheet is misaligned from my confusion matrix. Here is the cost matrix after realigning the columns so they match my confusion matrix:

0	5
1	0

Coupling with our confusion matrix, and summing, we get a total error cost (score where lower is better) of the model:

$$0(7) + 5(54) + 1(12) + 0(127) = 0 + 270 + 12 + 0 = 282$$

Limitations of the Model

There are a few reasons I think the model did so poorly in testing:

1. **`max_depth` of 4:** I chose to restrict my tree to a `max_depth` of 4 mainly to make it readable (presentable and understandable) for the assignment. I also did it to avoid overfitting to the training data (as by height of 4, a lot of leaf nodes had less than 10 values). This obviously lead to some problems as some leaf nodes still had over 100 data objects, meaning a deeper tree would have helped.
2. **Only using 5/20 attributes from original dataset:** Using only a subset of the descriptive features available means we likely missed the most 'useful' features. We are making uninformed guesses as the model has to predict the target value with only 25% of the information. Additionally, we only used categorical attributes, and numerical values such as `age` and `credit_amount` would likely have helped much more than `foreign_worker`, though that is my own assumption.
3. **`DecisionTreeClassifier` can construct binary trees only:** This means that the algorithm is restricted to making binary decisions only. One-hot encoding turns multi-category attributes into individual binary columns. Because the tree can only test one binary condition at a time, it takes multiple sequential splits just to evaluate a single original feature. Instead of analyzing all of a feature's categories simultaneously, the tree wastes depth splitting it up multiple times. This, paired with `max_depth` being 4 leads to a bad recipe for a model.
4. **Dataset being fake:** We've talked about this in the labs, but the German Credit dataset being fake means there aren't any realistic patterns and a lot of noise, meaning the model cannot extract any meaningful patterns that exist in the real world.

One solution would be increasing the `max_depth` but using pre-pruning to get rid of any leaf nodes with less than 10 attributes so we don't overfit, along with introducing some Gini/information gain threshold as well.

- *Note: I tried this and it still did not result in any meaningful improvement, the final cost score was still 277.*